

MES COLLEGE OF ENGINEERING, KUTTIPPURAM

THRIKKANAPURAM, MALAPPURAM DIST. KERALA



Assignment Records 2

Dept. computer Applications

Name: THANSIYA NASRIN A

Semester / Branch S2 MCA

Roll No. 59

Subject

20MCA102 ADVANCED DATABASE MANAGEMENT SYSTEMS.

	Submission Due Date	Submitted on	Marks
Assignment-1			
Assignment-2	28.04.2026	27.04.2026	
Assignment-3			
Assignment-4			
Assignment-5			
		Total Marks (2)	

Name and Signature of Faculty

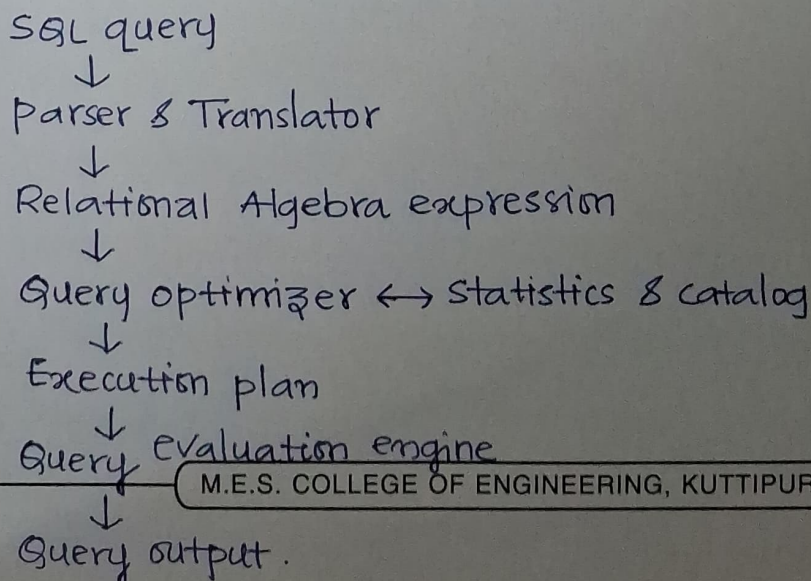
Q1. Explain query processing and discuss the steps involved in query processing with a clear diagram and analyze each step in detail.

Ans. Query processing refers to the set of activities involved in extracting data from a database in response to a high-level query (typically written in SQL). It transforms a user query into an efficient execution strategy that retrieves the correct results with minimal resource usage.

Steps in Query processing:

1. parsing and Translation: The SQL query is first checked for syntax errors and then translated into an internal representation, usually a relational algebra expression or a parse tree. This step ensures the query is ~~sy~~ semantically valid. (Eg: table and attribute names exist).
2. Query optimization: The system generates multiple equivalent execution plans for the query and estimates the cost of each (in terms of I/O, CPU, and memory). The plan with the lowest eliminated cost is selected. This is one of the critical steps.
3. Query Evaluation / Execution: The chosen query execution plan is carried out by the query execution engine. operators like select, join, and project are executed using specific algorithms, and the results are returned to the user.
4. Result Return: The final output of the query is formatted and presented to the user or application.

↳ Diagram of query processing pipeline:



Q2. List out and illustrate query evaluation strategies for Selection operation.

Ans. The selection operation (σ) retrieves tuples from a relation that satisfy a given condition. Several strategies exist, each with different performance characteristics depending on the data and available indexes.

1. Linear Search (Full Table Scan) : Every tuple in the relation is scanned and checked against the selection condition. This is the most basic method and works for any condition, but it is most expensive — cost = number of blocks (b).
2. Binary Search : Applicable when the file is ordered on the selection attribute. The search narrows down to the matching block in $O(\log_2 b)$ steps. It is efficient but requires a sorted file.
3. primary Index on key Attribute (Equality) : If there is a primary index on the key attribute used in the condition, the index is traversed to find the exact record.
cost = height of B+ tree + 1 block access.
4. primary Index on Ordering Non-key Attribute : Used when equality is on a non-key attribute. Multiple consecutive records may match, so after locating the first via index, subsequent blocks are scanned.
cost = index traversal + number of matching blocks.
5. secondary Index (Equality on Non-ordering key) : A secondary (dense) index is used to find matching records. Each matching record may be in a different block, so cost can be high if many records match —
cost = index levels + number of matching records.
6. Conjunctive/Disjunctive selection : For conditions with AND (conjunctive) or OR (disjunctive) operators, composite indexes or bitmap indexes may be used, or individual conditions are evaluated and their results intersected/unioned.

Q3. Discuss and evaluate the query processing cost of primary Index with equality on key and non-key attribute.

Ans. The cost of query processing using a primary Index is measured in terms of the number of block transfers and seeks required to retrieve matching records.

↳ primary Index on a key Attribute
(Equality condition):

When the selection condition is an equality condition on a key attribute (meaning at most one record matches), the process is straightforward and efficient.

- The primary Index (usually a B+ tree) is traversed from root to leaf to locate the pointer to the data block.
- Since the attribute is a key, only one record exists that satisfies the condition.
- $\text{cost} = (h_i + 1)$ block accesses, where h_i is the height of the Index tree and $+1$ is for the actual data block access.
- This is the most efficient case of primary Index-based retrieval.

↳ primary Index on a Non-key Attribute
(Equality condition):

When the selection is on a non-key attribute, multiple records can satisfy the condition, and since it's a primary (clustering) Index, matching records are stored in consecutive blocks.

- The Index is first traversed to find the first matching record's block.
- Then consecutive blocks are scanned until no more matching records found.
- $\text{cost} = h_i + \text{number of blocks containing matching records (b)}$.
- If many tuples match, this can be significantly more expensive than the key case.
- However, because the data is clustered (physically ordered), it is still more efficient than a secondary Index in this scenario.